

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	MD Razia
Student Name	P Tanvitha
Roll Number	2520030385
Branch	CSE
Course Outcome	CO-2

Case Study Topic: HDFC NetBanking Daily Spend Prefix-Sum Management using Fenwick Tree (Binary Indexed Tree)

Work Done Summary:

In this case study, we implemented a Fenwick Tree (Binary Indexed Tree) to efficiently manage HDFC NetBanking customers' daily spending records.

The system stores daily transaction amounts and supports two key operations:

1. **Point Update** – Add a transaction amount to a specific day's spending record.
2. **Range Sum Query** – Calculate total spending within a given date range for monthly statement generation.

We constructed the BIT array from the daily spend data and used prefix sums to answer range-sum queries efficiently. The solution provides $O(\log n)$ time complexity for both updates and queries, making it suitable for large-scale banking systems.

Implementation Details:

1. Stored daily transaction amounts in the array `spend[1..15]`.
2. Constructed the Fenwick Tree using cumulative range sums.
3. Each BIT node stores the sum of a specific range determined by $LSB(i)$.
4. Calculated `bit[4]` by summing `spend[1..4]`.
5. Calculated `bit[8]` by summing `spend[1..8]`.
6. Calculated `bit[12]` by summing `spend[9..12]`.
7. Used prefix sums to compute range queries: $\text{Range Sum}(l,r) = \text{Prefix}(r) - \text{Prefix}(l-1)$
8. Queried total spending from Jan 5 to Jan 12 using: $\text{Prefix}(12) - \text{Prefix}(4)$
9. Verified results with direct summation of daily transactions.
10. Observed logarithmic performance for updates and queries.

Result: Screenshots / Output

```
Run Main
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition\lib\idea_r.jar=S6008
HDFC NETBANKING DAILY SPEND MANAGEMENT USING FENWICK TREE (BIT)
INPUT SPEND ARRAY (1..15):
Index: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
Spend: 1200 800 0 2400 1500 600 0 0 3500 0 1100 950 700 0 0
FENWICK TREE (BIT) ARRAY (1..15):
Index: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
BIT : 1200 2000 0 4400 1500 2100 0 6500 3500 3500 1100 5550 700 700 0
bit[4] = 4400
bit[8] = 6500
bit[12] = 5550
VERIFICATION OF IMPORTANT BIT CELLS:
bit[4] covers indices [1..4] = 1200 + 800 + 0 + 2400 = 4400
bit[8] covers indices [1..8] = 1200 + 800 + 0 + 2400 + 1500 + 600 + 0 + 0 = 6500
bit[12] covers indices [9..12] = 3500 + 0 + 1100 + 950 + 5550
PREFIX SUM CALCULATIONS:
Visited BIT cells for prefix(12): bit[12] bit[8]
prefix(12) = 5550 + 6500 = 12050
Visited BIT cells for prefix(4): bit[4]
prefix(4) = 4400
RANGE QUERY (JAN 5-12):
Spend[5..12] = prefix(12) - prefix(4)
= 12050 - 4400
= 7650
VERIFICATION (DIRECT SUM):
1500 + 600 + 0 + 0 + 3500 + 0 + 1100 + 950 = 7650 ✓
TOTAL SPEND FROM JAN 5 TO JAN 12 = ₹7650
PROCESS FINISHED WITH EXIT CODE 0
```

Fenwick Tree efficiently supported both update and query operations, making it highly suitable for banking transaction analytics and monthly statement generation.

GitHub Link: <https://github.com/pondurutanvitha-stack/DSA-2>

Student Signature	Faculty Signature